



Code of Conduct:

I AKSHAYA SEKAR/1911260204 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Akshaya Sekar



Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Question 1: A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer: Testing should be treated as an engineering activity rather than an informal activity. For the banking application, the team should first analyze requirements and identify critical functions such as login, account balance, fund transfer, withdrawal, and transaction confirmation. A structured test plan should define scope, risks, responsibilities, entry and exit criteria, and expected results. Test cases should be designed systematically using equivalence partitioning, boundary value analysis, decision tables, state transition testing, and requirement-based testing instead of ad-hoc testing. Automation tools can be used for repeatable regression tests, while defect-management tools can record severity, priority, status, and traceability. Reviews, coding and testing standards, and a defined test lifecycle should be followed consistently. For example, a fund-transfer test can cover valid transfers, insufficient balance, invalid account numbers, transaction limits, and network interruptions. This approach improves coverage, makes testing measurable and repeatable, detects defects earlier, and improves the overall reliability and quality of the banking system.

Question 2: An ATM system produces inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects.



Conclude expected outcomes.

Answer: A structured testing process should cover the complete SDLC. During requirement analysis, the tester reviews ATM requirements for correctness and identifies ambiguous conditions. During design, test conditions and expected results are derived from withdrawal rules. During implementation, unit testing verifies individual modules such as PIN validation, balance checking, and cash-dispensing logic. Integration testing checks communication between the ATM, bank server, account database, and cash dispenser. System testing validates the complete withdrawal transaction, while acceptance testing confirms that the system satisfies user and business requirements. For an ATM withdrawal, test cases should include valid withdrawal, insufficient balance, invalid PIN, daily-limit violation, unavailable cash, cancelled transaction, and communication failure. Defects should be recorded and retested after fixes, followed by regression testing. Consistency across these stages reduces missed defects and improves traceability, reliability, and confidence before deployment. Expected outcomes include fewer transaction failures, better defect detection, and more dependable ATM operations.

Question 3: A web application shows a mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer: An error is a human mistake made during requirements, design, coding, or another development activity. A defect is the flaw introduced into the software because of that error. A failure occurs when the executed software produces an incorrect or unexpected result. For example, if a developer misunderstands the login requirement and writes an incorrect password-validation condition, the misunderstanding is an error, the incorrect condition in the program is a defect, and allowing an invalid password to access the account during execution is a failure. The same relationship can occur in transaction processing when the implemented transaction limit differs from the specified requirement. Confusing these terms can produce unclear defect reports and make root-cause analysis difficult. Testers should report the observed failure with clear steps, input data, expected result, actual result, environment, severity, and supporting evidence. Developers can then locate and correct the underlying



defects. Clear terminology improves communication, traceability, and the effectiveness of the testing and debugging process.

Question 4: A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer: Defect clustering means that a relatively small number of modules often contain a large proportion of the defects. If defects are concentrated in a few modules, the testing team should use historical defect data and risk information to give those modules greater testing attention. Test planning should identify high-risk and frequently failing components and allocate more test cases, reviews, and regression tests to them. Test cases can be prioritized according to business criticality, defect history, complexity, frequency of change, and impact of failure. Other principles such as early testing, exhaustive testing being impossible, and the need for varied test techniques should also guide the plan. For example, if an online banking application has most previous defects in fund transfer and authentication modules, these areas should receive deeper functional, boundary, negative, security, and regression testing than low-risk informational pages. This approach uses resources efficiently, increases the probability of finding important defects early, and improves overall test effectiveness without testing every component equally.

Question 5: A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect

Answer: Defects originate at different stages such as requirements, design, coding, integration, or configuration. They can be categorized by origin, severity, priority, type, and affected functionality. A defect repository should record details such as defect ID, module, description, steps to reproduce, expected and actual results, severity, priority, status, root cause, and resolution. The existing repository should be analyzed to identify repeated modules, defect patterns, common root causes, and defects that escaped into production. This information can be converted into new regression and targeted test cases. For example, if previous releases repeatedly contain validation defects in a payment module, future releases should include stronger positive, negative, boundary, and invalid-input tests for that module.



Root-cause analysis and preventive actions should also be performed rather than simply fixing individual defects. Effective repository use improves traceability, prevents recurrence, strengthens test design, and gradually improves software quality across releases.

Question 6: A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer: For a mobile banking fund-transfer crash, the tester should first reproduce the failure and collect logs, input conditions, device and operating-system details, network conditions, and transaction information. The functionality should then be tested systematically using valid and invalid accounts, insufficient balance, transaction limits, duplicate requests, network interruption, timeout, session expiry, and rapid repeated transfers. Boundary value analysis and negative testing can reveal conditions that ordinary tests miss. Integration testing should verify communication among the mobile application, authentication service, transaction service, and database. Regression testing should be performed after every fix. If the crash occurs only under a particular network or device condition, compatibility and reliability testing should also be included. Defects should be documented clearly with reproducible steps and evidence, and root-cause analysis should be used to identify the underlying problem rather than only treating the visible symptom. These practices reduce repeated failures and improve the reliability of mobile banking transactions.

Question 7: A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer: Testing and debugging are related but different activities. Testing is the process of executing or evaluating software to find defects and determine whether it meets specified requirements. Debugging is the development activity of locating the cause of a known defect, correcting the code or configuration, and verifying the correction. For example, if a banking application displays an incorrect balance, a tester can reproduce the failure, record the input and expected result, and report the defect. The developer then debugs the application by tracing the calculation or database logic, identifying the root cause, applying a correction, and



retesting it. If a team treats debugging as testing, it may focus only on fixing known problems and miss other defects. A structured approach is therefore to separate independent testing from debugging, maintain clear defect reports, perform developer-level correction, and then conduct tester verification and regression testing. This division improves objectivity, accountability, defect detection, and software quality.

Question 8: An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer: Cross-browser testing should verify that the e-commerce application provides consistent functionality, appearance, and performance across supported browsers and operating systems. The team should first identify the browser and device combinations used by the target users and prioritize them according to usage and risk. Test cases should cover login, product search, cart operations, checkout, payment, navigation, forms, cookies, sessions, and responsive layouts. Defects may originate from differences in browser rendering engines, JavaScript support, CSS behavior, APIs, or browser-specific configuration. A systematic strategy can use a combination of real devices, virtual machines, and browser automation tools such as Selenium or Playwright, where appropriate. The same requirement-based regression suite should be executed across the selected browser matrix, with screenshots and logs attached to defects. Compatibility, functionality, responsiveness, and regression testing together provide better coverage. This approach reduces browser-specific failures and improves reliability and customer experience.

Question 9: A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer: The incorrect salary calculations are primarily coding or functional defects because the implemented program logic produces a result that does not satisfy the payroll requirements. The tester should trace the defect from the requirement and design through the implemented calculation rules, including basic salary, allowances, deductions, tax, overtime, and rounding. Classification helps because it identifies the nature and likely source of the problem and allows suitable tests to be selected. Test cases should include normal values, zero



values, minimum and maximum salary values, boundary tax ranges, overtime conditions, deductions, and combinations of allowances. Reviews and unit tests can detect incorrect formulas before integration, while automated regression tests can ensure that later changes do not reintroduce the problem. Requirements should also be made precise so that expected calculations are unambiguous. Proper classification, systematic test design, code review, unit testing, and regression testing can reduce repeated payroll calculation defects in future releases.

Question 10: A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer: Random testing alone does not provide controlled coverage, so the team should redesign testing as an engineering activity. The first step is to understand and trace requirements to test conditions and test cases. Test cases should have a unique identifier, objective, preconditions, input data, steps, expected results, and pass or fail criteria. Suitable techniques include equivalence partitioning, boundary value analysis, decision tables, state transition testing, use-case testing, and risk-based testing. For example, for an online payment amount field, partitions can include valid amounts, zero, negative values, values above the permitted limit, and invalid formats; boundary values should also be tested. Test cases should be reviewed, executed systematically, and maintained for regression testing. Automation can be used for repeatable checks, while exploratory testing can supplement planned tests. Structured design improves coverage, repeatability, traceability, and defect detection and makes the testing process more efficient and measurable.

Question 11: A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer: Data inconsistency across healthcare modules can result from incorrect mapping, integration defects, synchronization problems, inconsistent validation, database transaction issues, or incorrect assumptions between modules. The tester should first identify the source and flow of important data, such as patient identity, medication, laboratory results, and appointment information. Integration and end-to-end test cases should verify that data entered

in one module is transferred accurately and remains consistent in connected modules.

Boundary, negative, duplicate data, concurrent-update, and failure-recovery scenarios should also be tested. Requirements and interface specifications should be traced to test cases so that expected values are clear. Every defect should be recorded with the affected module, data values, steps to reproduce, expected result, actual result, severity, and evidence. Regression testing should be repeated after corrections. Systematic integration testing, data validation, traceability, and disciplined defect reporting help protect data integrity and improve reliability of the healthcare application.

Question 12: A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect

Answer: Repeated defects indicate that the organization is not fully learning from previous failures. The defect repository should be analyzed using fields such as module, defect type, severity, root cause, release, detection phase, and recurrence. Patterns can then be identified, for example, repeated validation defects in the same module or repeated integration defects after particular changes. These patterns should be converted into additional regression tests and risk-based test scenarios. Root-cause analysis should determine whether the underlying problem is unclear about requirements, design weakness, coding practices, inadequate reviews, missing tests, or configuration errors. The repository should also record resolution and preventive action so that future testers can reuse the knowledge. For example, if every release produces a defect in transaction-limit validation, boundary and negative tests for those limits should become permanent regression tests. Regular repository reviews and trend analysis help prevent recurrence, improve test coverage, and strengthen the overall testing strategy.

Question 13: A web-based learning system fails during peak usage due to untested scenarios.

Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer: The peak-usage failure indicates that the learning system did not adequately test realistic load and stress conditions. Test planning should identify performance as a quality requirement and include normal load, peak load, stress, endurance, and recovery scenarios where applicable. The team should determine expected response time, concurrent users, throughput, and acceptable error rates. Performance test cases should represent realistic activities such as login, course access, video requests, assignment submission, and result of viewing occurring simultaneously. Test coverage should also include boundary conditions such as maximum concurrent users and limited server resources. Monitoring CPU, memory, database activity, response time, throughput, and error rates helps identify bottlenecks. Automated performance tools can generate repeatable load, while functional regression tests ensure that performance changes do not break core behavior. Better planning and coverage expose capacity problems before release, allowing the team to tune infrastructure or application components and improve reliability during peak usage.

Question 14: A team observes that testers are not involved in early development stages.

Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer: Testers should be involved from the beginning of the SDLC rather than only after coding is complete. During requirements analysis, testers can identify ambiguity, missing acceptance criteria, inconsistent rules, and untestable requirements. During design, they can review interfaces, workflows, risks, and testability. During implementation, they can prepare test cases and support unit and integration testing. During system testing, they execute functional, non-functional, regression, and exploratory tests and report defects. Early involvement follows the principle that testing should begin as early as possible because defects are generally easier and less costly to correct near their source. For example, if a tester identifies an ambiguous payment rule during requirements of review, the requirement can be clarified before developers implement incorrect logic. Teams should use requirement reviews,

shared test plans, regular developer-tester discussions, and common defect-tracking tools. Early collaboration improves communication, reduces rework, and increases product quality.

Question 15: A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer: Incomplete test coverage means that important conditions, paths, input classes, or interactions have not been adequately tested. The team should begin by tracing requirements to test conditions and identifying untested areas using coverage reports, risk analysis, and defect history. Equivalence partitioning can cover representative input groups, while boundary value analysis targets values at and around limits. Decision tables are useful for combinations of business rules, and state transition testing is useful for systems whose behavior changes according to state. Negative and error-handling scenarios should also be included. For example, a transaction system should test valid transactions as well as insufficient balance, limits, duplicate requests, cancelled transactions, timeouts, and communication failures. High-risk and frequently changed modules should receive additional regression testing. Proper coverage does not mean testing every possible input; it means selecting systematic tests that cover important requirements and risks. Improved test design increases confidence and reliability.

Question 16: A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help track. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer: Effective defect reporting should provide enough information for another team member to reproduce and understand the problem. A structured report should contain a unique defect ID, concise title, affected module, environment, build version, preconditions, exact steps, test data, expected result, actual result, severity, priority, attachments, and current status. Clear definitions are important because an error is a human mistake; a defect is the resulting flaw in a work product, and a failure is the incorrect behavior observed during execution. For example, instead of reporting 'payroll is wrong,' the tester should specify the employee data used, the expected salary calculation, the actual amount, and the steps that produced it. A standard template and centralized defect repository improve traceability and communication. Developers can reproduce and fix defects faster, testers can verify fixes, and managers can

track trends. Clear reporting therefore reduces delays and supports better quality management.

Question 17: A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer: Process-based testing uses defined activities, responsibilities, inputs, outputs, and standards so that testing is performed consistently across modules. The organization should establish a common test lifecycle covering requirement analysis, planning, test design, environment preparation, execution, defect reporting, retesting, regression testing, and closure. Standard templates should be used for test cases, test plans, and defect reports. Requirements should be traced to test cases so that each module receives appropriate coverage. Reviews and common entry and exit criteria should be applied across teams, while automation can handle repeatable regression checks. For example, if several modules contain input forms, each team can follow the same rules for positive, negative, boundary, validation, and error-message testing. Metrics such as pass rate, defect density, coverage, and defect leakage can be reviewed for consistency. Standardization reduces variation between testers, improves traceability and repeatability, and produces more predictable quality outcomes.

Question 18: A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer: Edge cases are often missed when testers focus only on normal inputs and common user flows. The team should review requirements to identify minimum, maximum, just-below, just-above, empty, zero, duplicate, invalid, and unexpected values. Boundary value analysis is particularly useful because defects frequently occur at the edges of valid input ranges. For example, if an age field accepts values from 18 to 60, tests should include 17, 18, 19, 59, 60, and 61, along with invalid formats where appropriate. The same principle can be applied to transaction amounts, file sizes, dates, user counts, and other limits. Edge cases should be included in the test design before execution and retained in the regression suite. Risk-based testing can give greater priority to boundaries affecting critical functions. This approach improves defect detection and reduces the chance that failures will appear only after deployment.

Question 19: A company lacks proper documentation of defects, making it difficult to track

issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer: A defect repository provides a central, traceable record of software defects throughout their lifecycle. To address missing documentation, the organization should establish a standard repository with fields such as defect ID, title, description, module, release, environment, steps to reproduce, expected and actual results, severity, priority, owner, status, root cause, resolution, and verification result. Each defect should be updated as it moves from new to assigned, fixed, retested, and closed, or reopened when necessary. Structured records allow testers and developers to search for previous problems, identify recurring defects, and analyze trends by module or defect type. For example, if several releases show repeated validation defects in one component, the team can create targeted regression tests and review the component's requirements and design. Repository data also supports management reporting and process improvement. Good documentation improves traceability, communication, accountability, root-cause analysis, and long-term defect prevention.

Question 20: A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves

Answer: The banking system should adopt a structured, risk-based testing process rather than relying on inconsistent testing. First, requirements should be reviewed, and critical transaction flows such as login, balance inquiry, transfer, withdrawal, and payment should be identified. A test plan should define scope, risks, environments, responsibilities, entry and exit criteria, and test levels. Test cases should be designed using equivalence partitioning, boundary value analysis, decision tables, state transitions, negative testing, and risk-based prioritization. Integration testing should verify communication between the application, authentication services, transaction services, and databases. Regression testing should be automated for stable, repeatable transaction scenarios. Every defect should be recorded with clear evidence and severity, followed by retesting and regression after correction. For example, a transfer test suite should include valid transfers, insufficient funds, invalid accounts, transaction limits, duplicate requests, timeouts, and interrupted connections. Structured testing improves traceability, coverage, early defect detection, reliability, and confidence in banking transactions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand